DSA 495 601 Project Milestone 5

Thomas Grover

- Introduction
- Setup
- 1. Train and validate learners
- 2. Distribution of predicted probabilities
  - 2.1 Overall distribution by learner
  - 2.2 Distribution split by true outcome
- 3. Discrimination metrics
  - 3.1 ROC curves
  - 3.2 Precision-Recall curves
  - 3.3 Cross-validated performance summary
- 4. Calibration
- 5. Classification performance at different thresholds
  - 5.1 Select thresholds to test
  - 5.2 Key classification metrics
  - 5.3 Calculate metrics for all learners
  - 5.5 Visualize confusion matrix statistics
- 6. Fairness assessment
  - 6.1 Distribution of predictions by group
  - 6.2 Calibration by group
  - 6.3 Classification metrics by group
- 7. Results summary
- 8. Final learner selection
  - Step 1: Narrow down candidates
  - Step 2: Consider simplicity and tranparency
  - Step 3: Make your selection
  - Step 4: Justify your choice
- 9. Save results for testing
- Appendix: Advanced threshold selection (Optional)
  - A. Distribution-based thresholds
  - B. Criteria-based thresholds

Introduction

For Milestone 5, you will train your specified learners, evaluate their performance, and select your final model for testing. This milestone focuses on understanding model performance from multiple perspectives: discrimination, calibration, and fairness.

Your goals:

1. Train all learner configurations from Milestone 4
2. Evaluate discrimination using ROC and PR curves
3. Assess calibration of predicted probabilities
4. Examine classification metrics at different thresholds
5. Investigate fairness across demographic groups

6. Select and justify your final learner

Important notes:

- Model training may take 15-60+ minutes depending on your specifications and data size
- All coding you will need to modify or expand are marked with @TODO
- Answer all questions marked with @TODO

After completing all sections, knit to HTML, then download and print/save as PDF using this naming convention: `DSC495_601_FA25_Milestone5_unityID.pdf`

Due date: November 6 11:45 a.m. ET

Setup

This section loads all necessary files and configures your R environment for model training.

```
# Set random seed for reproducibility
seed <- 2024

# Configure notebook settings
knitr::opts_chunk$set(
  cache = FALSE,      # Don't cache results
  warning = FALSE,    # Suppress warnings
  message = FALSE,    # Suppress messages
  fig.width = 8,
  fig.height = 7
)

# Load helper scripts with custom functions
# These must be in your R/ subdirectory
source("/share/dsa495601f25/templates/R/packages.R")
source("/share/dsa495601f25/templates/R/learner_training.R")
source("/share/dsa495601f25/templates/R/learner_training_helpers.R")
source("/share/dsa495601f25/templates/R/equity_metrics.R")
source("/share/dsa495601f25/templates/R/performance_metrics.R")
source("/share/dsa495601f25/templates/R/plots_updated.R")

# Enable parallel processing for faster training
library(doParallel)
n_cores <- parallel::detectCores() - 1
cl <- makePSOCKcluster(n_cores)
registerDoParallel(cl)
cat("Parallel processing enabled with", n_cores, "cores\n")
```

```
## Parallel processing enabled with 31 cores
```

```r
# Load your processed data from Milestone 3
trainDat <- readRDS(paste0(params$my_path, "trainDat.rds"))
testDat  <- readRDS(paste0(params$my_path, "testDat.rds"))

# Load learner specifications from Milestone 4
learnerSpec <- readRDS(paste0(params$my_path, "learnerSpec.rds"))

# Load preprocessing recipes from Milestone 4
recipes <- readRDS(paste0(params$my_path, "recipes.rds"))

# mainRecipes <- readRDS(paste0(params$my_path, "recipe_main.rds"))

# Verify everything loaded correctly
cat("\nData verification:\n")
```

```
##
## Data verification:
```

```r
cat("Training data:", nrow(trainDat), "rows,", ncol(trainDat), "columns\n")
```

```
## Training data: 31647 rows, 73 columns
```

```r
cat("Testing data:", nrow(testDat), "rows,", ncol(testDat), "columns\n")
```

```
## Testing data: 13564 rows, 73 columns
```

```r
cat("Predictor sets:", length(learnerSpec$allPredsets), "\n")
```

```
## Predictor sets: 4
```

```r
cat("Outcome variable:", learnerSpec$outcomeName, "\n")
```

```
## Outcome variable: y_binary
```

```r
cat("Positive class proportion:", round(learnerSpec$positiveProportion, 3), "\n")
```

```
## Positive class proportion: 0.117
```

---

1. Train and validate learners

In this section, you'll train all models specified in Milestone 4 using cross-validation. This is the most time-intensive step.

What happens during training:

- Each model trains on different subsets of your data (k-fold cross-validation)
- Hyperparameters are automatically tuned to find optimal settings
- Performance metrics are calculated for each model
- Predicted probabilities are saved for every observation

Expected computation time:

- Simple models (glm, lasso): 2-5 minutes
- Tree-based models (random_forest, xgboost): 15-60 minutes
- Total time = number of predictor sets × algorithms × folds
- For testing: You may want to try few models (temporarily edit Milestone 4 to include fewer ML algorithms)
- For frustrating, long run times: You can reduce grid_size in Milestone 4 (e.g. from 5 to 2 or 3)

Recommendation: Ensure you have at least 30-60 minutes available before running this code. Maximize the number of processors when you launch the HPC.

```
# Load results from previous run if desired
if (params$load_run==TRUE) learnerResults <- readRDS(paste0(params$my_path, "learnerResults.rd
s"))

if (params$load_run==FALSE) {
# Record start time
start_time <- Sys.time()
cat("Training started at:", format(start_time, "%H:%M:%S"), "\n\n")

# Train all learners using cross-validation
# This function handles all training, tuning, and validation
learnerResults <- train_learners(
  trainDat = trainDat,
  learnerSpec = learnerSpec,
  recipes = recipes,
  mainRecipes = mainRecipes
)

# Record end time and display duration
end_time <- Sys.time()
duration <- round(difftime(end_time, start_time, units = "mins"), 1)
cat("\nTraining completed at:", format(end_time, "%H:%M:%S"), "\n")
cat("Total training time:", duration, "minutes\n")

# Save training/validation results
saveRDS(learnerResults, file = paste0(params$my_path, "learnerResults.rds"))
}
```

```
## Training started at: 22:39:29
```

```
## finished training and validation for  glm_predSet_bm
## finished training and validation for  lasso_predSet_bm
```

```
## finished training and validation for  glm_predSet_extended1
## finished training and validation for  lasso_predSet_extended1
```

```
## finished training and validation for  random_forest_predSet_extended1
```

```
## finished training and validation for  xgboost_predSet_extended1
```

```
## finished training and validation for  glm_predSet_extended2
## finished training and validation for  lasso_predSet_extended2
```

```
## finished training and validation for  random_forest_predSet_extended2
```

```
## finished training and validation for  xgboost_predSet_extended2
```

```
## finished training and validation for  glm_predSet_kitchensink
## finished training and validation for  lasso_predSet_kitchensink
```

```
## finished training and validation for  random_forest_predSet_kitchensink
```

```
## finished training and validation for  xgboost_predSet_kitchensink
##
## Training completed at: 22:50:09
## Total training time: 10.7 minutes
```

```
# Extract key results
modelResults <- learnerResults$modelResults  # Cross-validated metrics
predProbs <- learnerResults$predProbs        # Predicted probabilities

# Display summary
cat("\n=== Training Summary ===\n")
```

```
##
## === Training Summary ===
```

```
cat("Number of learners trained:", length(table(predProbs$learnerName)), "\n")
```

```
## Number of learners trained: 14
```

```
cat("Number of observations:", nrow(trainDat), "\n")
```

```
## Number of observations: 31647
```

Checkpoint: Verify that training completed successfully. You should see: - Training time in minutes - Number of learners matching your Milestone 4 specifications - No error messages

---

2. Distribution of predicted probabilities

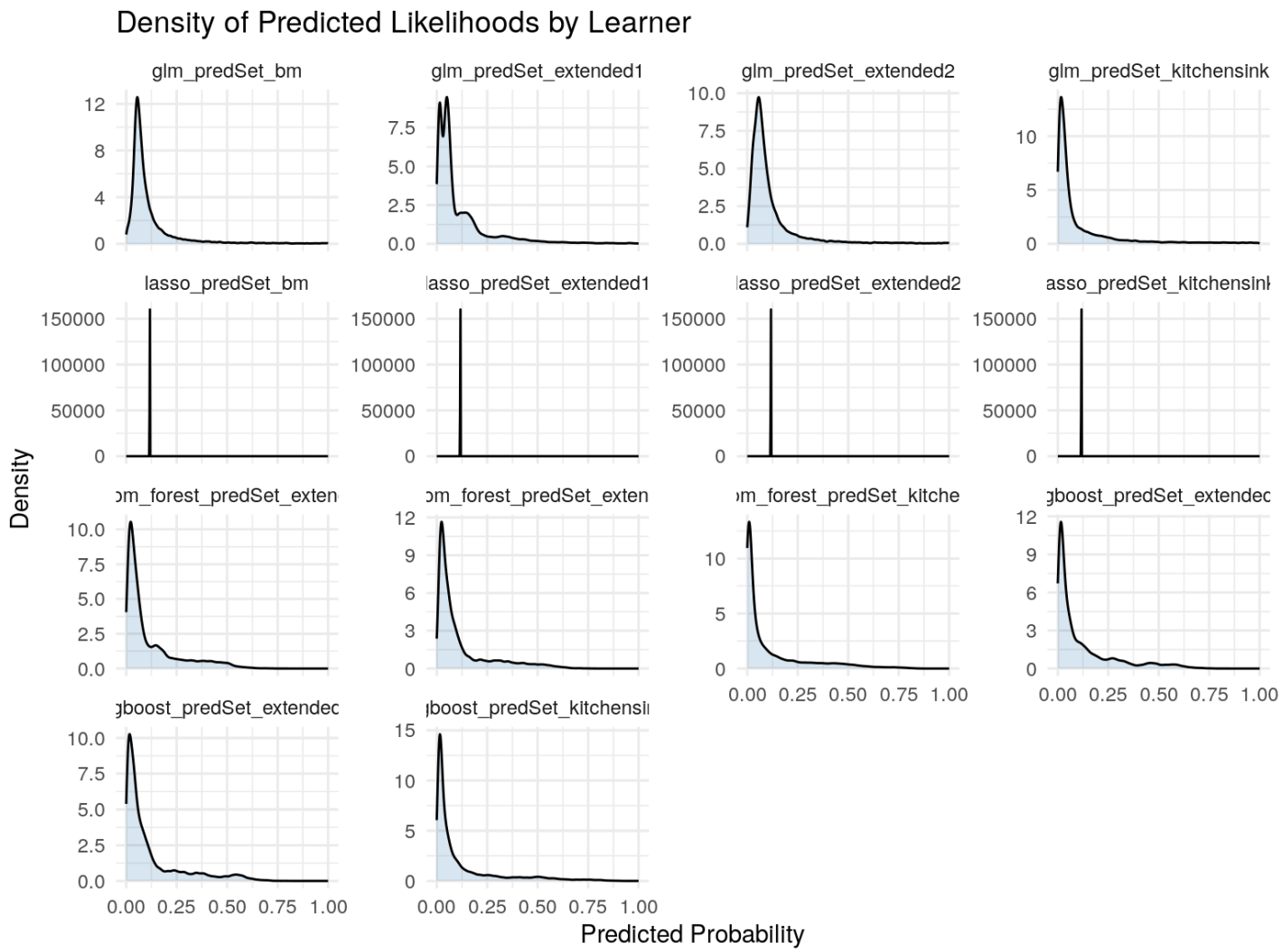
These plots show how your models distribute predicted probabilities across observations. Well-performing models should show clear separation between positive and negative outcomes.

2.1 Overall distribution by learner

This plot shows the overall distribution of predicted probabilities for each model.

What to look for: - Where do most predictions fall? - Do different models show different patterns? - Are predictions too clustered (limited range)?

```
# Plot predicted probability distributions for all learners
plot_predicted_probabilities(predProbs)
```



Question 1: Examine the predicted probability distributions. Which learner(s) show the widest range of predicted probabilities? Which show the narrowest range? What does the range of predictions tell you about a model's ability to discriminate between positive and negative cases?

The machine learners show the widest range of predicted probabilities, spanning from 0-1. The GLM learners show a wider range than Lasso, but not as wide as the machine learners. Lasso is by far the narrowest, likely because it is struggling to notice distinct differences between classes. The lasso model is very likely to predict a negative outcome.

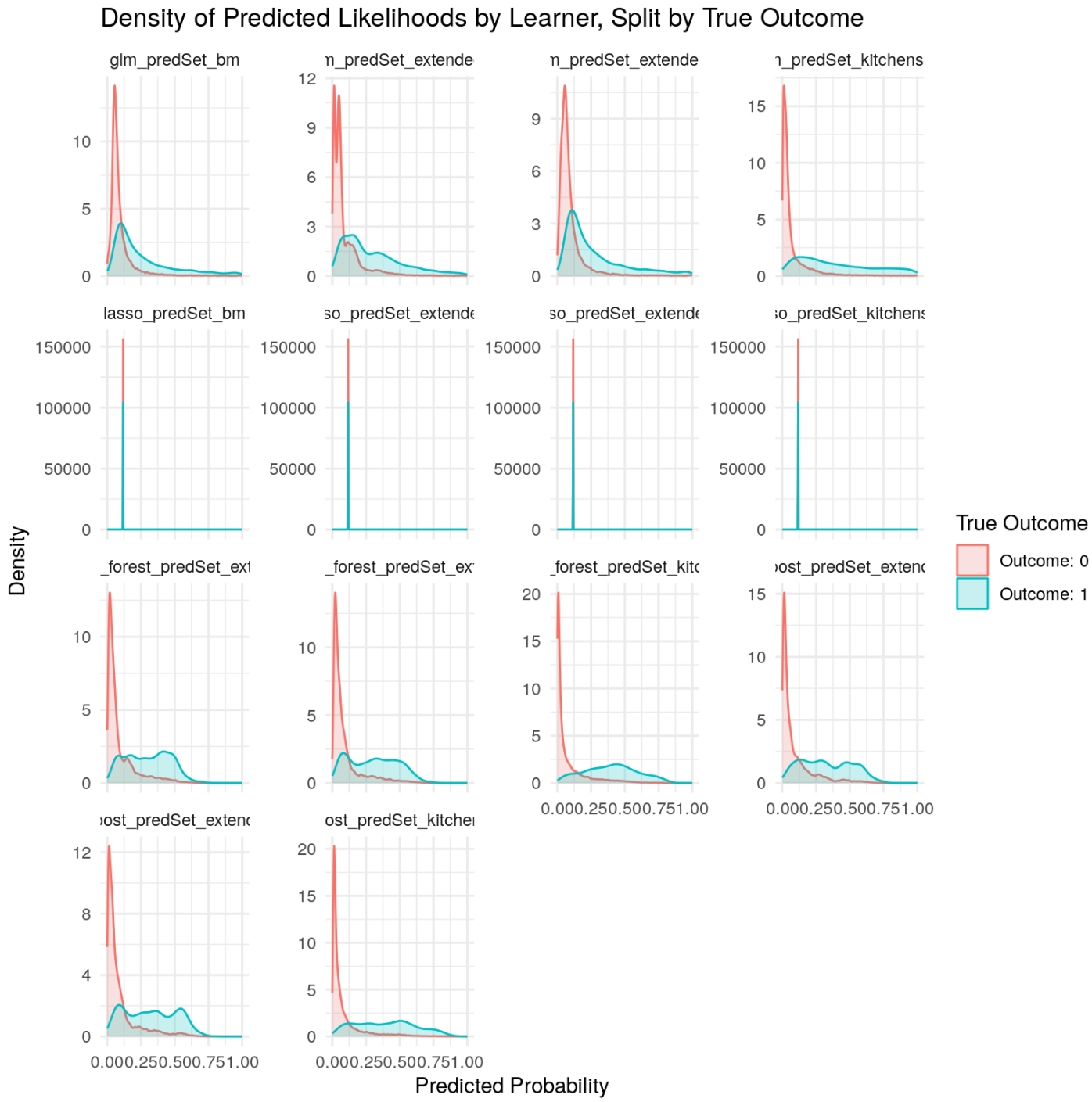
The range gives us insight into how effectively the model can discriminate positive and negative cases. A wider range indicates that the learner is better able to discriminate positive and negative cases.

2.2 Distribution split by true outcome

These plots show predicted probabilities separately for true positive (outcome=1) and true negative (outcome=0) cases.

What to look for: - Good separation between the two distributions = better discrimination - Overlapping distributions = model struggles to distinguish classes - Compare patterns across different learners

```
# Plot predicted probabilities split by actual outcome
plot_predicted_probabilities_split(predProbs, learnerSpec$outcomeName)
```



Question 2: For each learner, compare the distributions for outcome=0 (negative) vs outcome=1 (positive). Which learner shows the best separation between the two distributions? Which shows the most overlap? When distributions overlap significantly, what does this tell you about the model's ability to distinguish between classes?

The models that show the best separation are the Machine Learning Models, but what I find interesting is how much more effective the kitchen sink predictor set is than the others at discriminating outcomes. The Lasso distributions are almost completely overlapping, which suggests they are not very useful for discriminating positive and negative outcomes. It seems the lasso learners provide the same probability, almost treating all cases as negative.

The kitchensink predictor set performed the best across all the model types, which is an important observation. My main takeaway from these graphs is how comparable GLM_kitchensink is with the Machine Learners using the kitchen sink predictor set. We talked in lecture about how we want to use the most simple, translatable, and

reusable model that still predicts with an accuracy we are happy with. I think in this case, that might be GLM_kitchensink.

---

3. Discrimination metrics

Discrimination measures how well models distinguish between positive and negative cases. We evaluate this using ROC and PR curves.

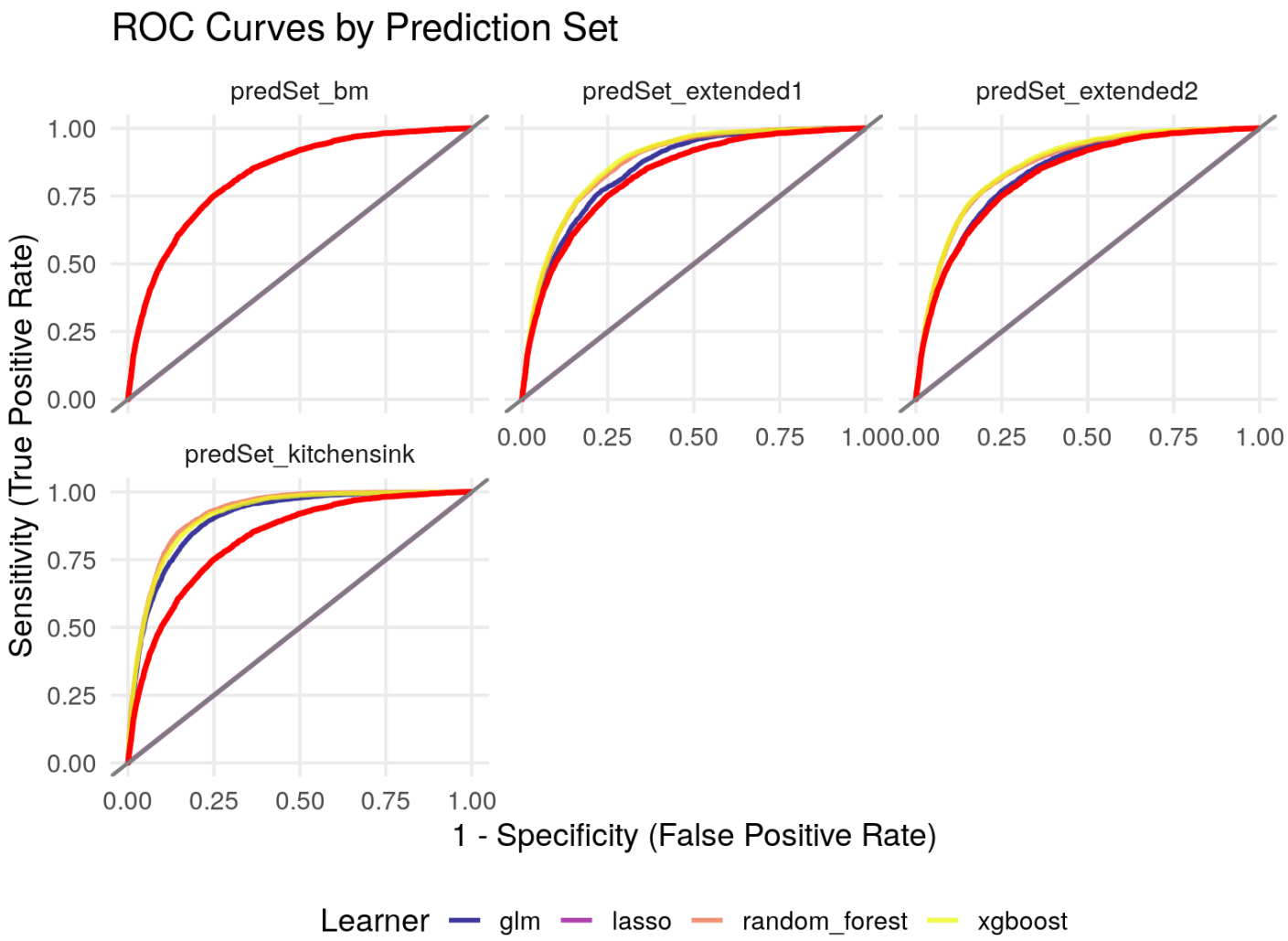
3.1 ROC curves

ROC (Receiver Operating Characteristic) curves show the trade-off between sensitivity (catching true positives) and specificity (avoiding false positives).

How to read the plot: - Curves closer to the top-left corner = better performance - Diagonal line = random guessing - The red line is your benchmark model for comparison

Important note for imbalanced outcomes: ROC curves can be overly optimistic when outcomes are rare because they give equal weight to both classes. A model can achieve high specificity simply by predicting “negative” most of the time, making the ROC curve look good even if it rarely catches true positives. This is why PR curves (next section) are more informative for imbalanced data.

```
# Plot ROC curves with benchmark reference
plot_roc_curve_with_reference(
  learnerResults$rocResults,
  referencePredSet = "predSet_bm"
)
```

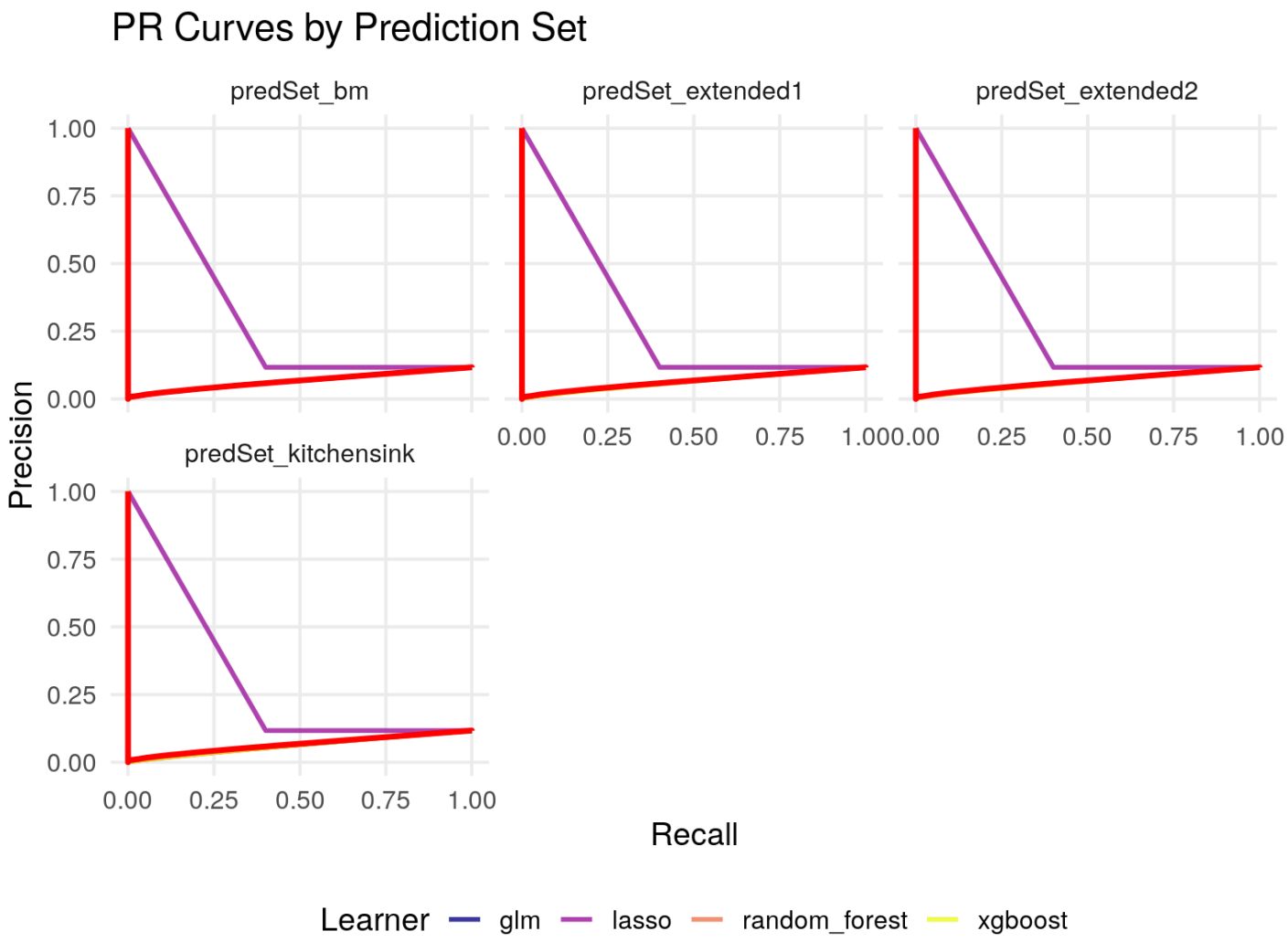


3.2 Precision-Recall curves

PR curves are especially useful for imbalanced outcomes (like yours with 11.7% positive rate). They show the trade-off between precision (accuracy of positive predictions) and recall (catching all true positives).

How to read the plot: - Curves closer to the top-right corner = better performance - PR-AUC is typically lower than ROC-AUC for imbalanced data - Focus on this metric more than ROC for rare outcomes

```
# Plot Precision-Recall curves with benchmark reference
plot_pr_curve_with_reference(
  prResults = learnerResults$prResults,
  referencePredSet = "predSet_bm",
  referenceLearner = "glm"
)
```



Question 3: Compare performance across both the ROC curves (above) and PR curves. Which learner(s) have curves closest to the ideal (top-left for ROC, top-right for PR)? Are the rankings of learners similar or different between the two types of curves? Given your outcome prevalence (shown in the Setup section), which type of curve (ROC or PR) should you focus on more for evaluating your models and why? Based on the curve type you've chosen to prioritize, do the more complex predictor sets show visually better performance than the benchmark, or do they appear quite similar?

The learners closest to ideal curves were the machine learning models. This can be easily determined in the ROC curves. The rankings of learners are similar in the 2 types of curves, but it can be difficult to distinguish what is really happening in the PR curves. We can clearly see lasso, starting high in precision with low recall. This essentially signals that the lasso model is very good at predicting negatives, but only when catching very few positive cases. We see the curve eventually return to baseline. The rest of the models essentially hover around baseline, which shows there is not much predictive power in the bank-marketing dataset.

With the imbalance of our target variable, the PR curve is the more valuable of the 2 because the ROC curve can make us think our model is over-performing. Because there are few positive cases, the ROC curve becomes inflated because even if it predicted all negatives, it would still be alright. This makes the ROC curve more useful when the data is more evenly spread. It is very difficult to determine in the PR curves, but if we take in the ROC curves, previous knowledge, and what we know about PR curves, it is reasonable to think that these more complex predictor sets result in better performance.

3.3 Cross-validated performance summary

This table shows the average discrimination performance across all cross-validation folds.

```
# Display cross-validated performance metrics
cat("=== Cross-Validated Performance Metrics ===\n\n")
```

```
## === Cross-Validated Performance Metrics ===
```

```
modelResults %>%
  select(learnerName, .metric, mean, std_err) %>%
  pivot_wider(names_from = .metric, values_from = c(mean, std_err)) %>%
  arrange(desc(mean_pr_auc)) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
## # A tibble: 14 × 5
##    learnerName           mean_pr_auc mean_roc_auc std_err_pr_auc std_err_roc_auc
##    <chr>                       <dbl>        <dbl>          <dbl>           <dbl>
##  1 random_forest_predSe…       0.592        0.922          0.012           0.002
##  2 xgboost_predSet_kitc…       0.579        0.916          0.012           0.003
##  3 lasso_predSet_bm            0.558        0.5            0               0
##  4 lasso_predSet_extend…       0.558        0.5            0               0
##  5 lasso_predSet_extend…       0.558        0.5            0               0
##  6 lasso_predSet_kitche…       0.558        0.5            0               0
##  7 glm_predSet_kitchens…       0.545        0.904          0.018           0.003
##  8 xgboost_predSet_exte…       0.464        0.877          0.007           0.004
##  9 random_forest_predSe…       0.451        0.873          0.007           0.004
## 10 xgboost_predSet_exte…       0.448        0.866          0.006           0.004
## 11 random_forest_predSe…       0.442        0.861          0.006           0.004
## 12 glm_predSet_extended1       0.423        0.851          0.01            0.003
## 13 glm_predSet_extended2       0.405        0.837          0.011           0.004
## 14 glm_predSet_bm              0.398        0.828          0.01            0.004
```

Question 4: Review the mean PR-AUC and ROC-AUC values with their standard errors. First, identify the 2-3 models with the highest mean values on your prioritized metric. Now look at the standard errors (std_err) - these indicate the uncertainty in the estimates. A rough rule of thumb: if two models' means differ by less than 2 times their standard errors, the difference may not be meaningful. For example, if Model A has mean=0.75 (std_err=0.03) and Model B has mean=0.73 (std_err=0.03), the difference of 0.02 is less than 2×0.03=0.06, suggesting the models perform similarly. Are the differences between your top models meaningful by this criterion, or are they essentially tied?

The 2 models with the highest PR-AUC are the random forest and XGBoost models for the kitchensink predictor set. Interestingly, the 4 lasso models are tied for 3rd place, and are actually not significantly different than the XGBoost model. From other context (discrimination, separation), we know these lasso models are not effective for predicting outcomes in the bank-marketing data, but I thought this was interesting, and shows the importance of looking at the big picture when selecting a model. Also of note, there is no statistically significant differene between the random forest and XGBoost models, so our top 2 are comparable.

The one other thing I wanted to point out was the effectiveness of the GLM_kitchensink learner. It was close behind the lasso models in PR curve, but it had a strong ROC curve as well. As a matter of fact, there is no statistically significant difference between the GLM_kitchensink, and the XGBoost_kitchen sink when looking at PR-AUC. I thought it was cool to see how well the GLM model held up compared to the more modern machine learning models.

---

4. Calibration

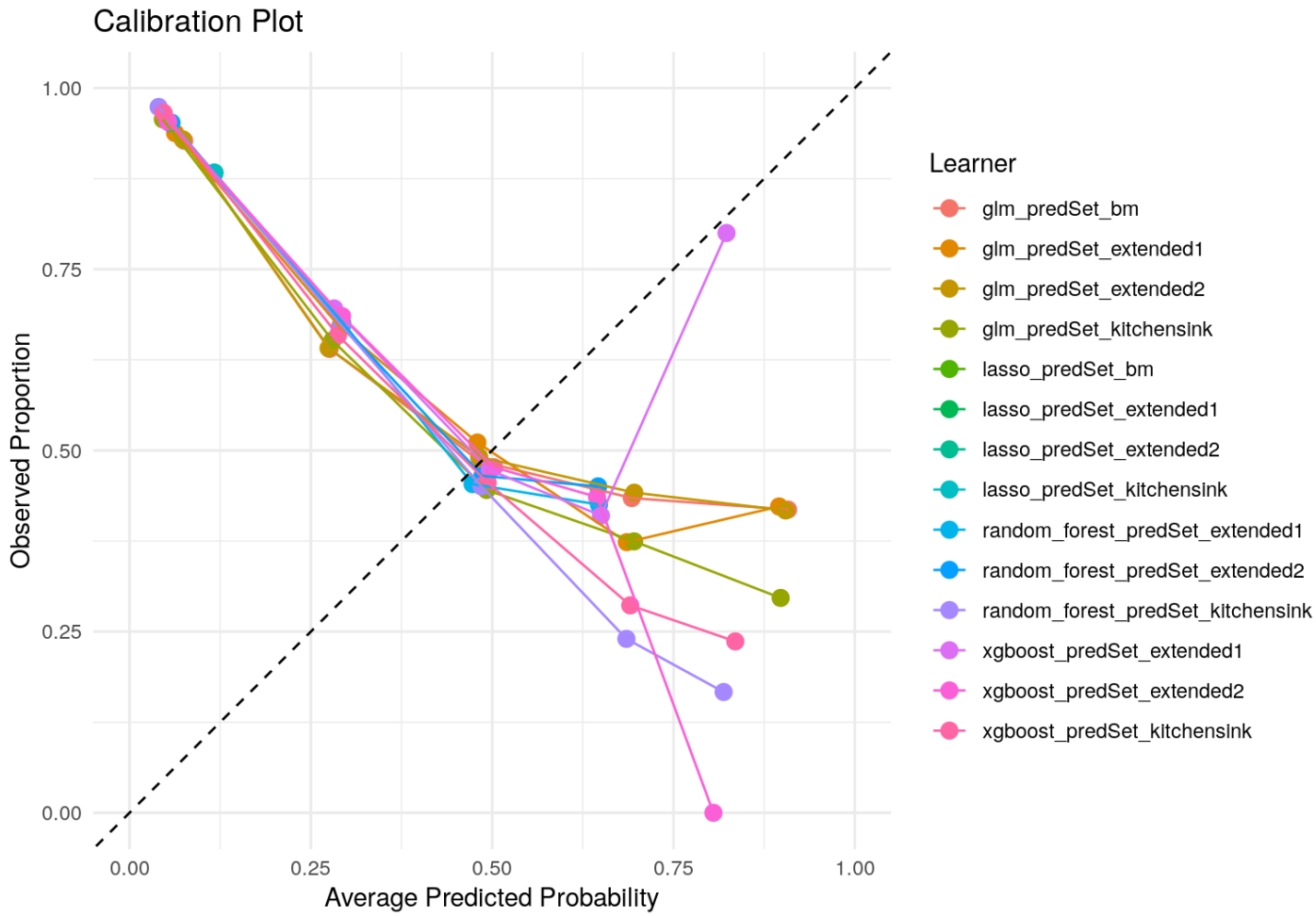
Calibration measures whether predicted probabilities match reality. A well-calibrated model that predicts 20% probability should be correct about 20% of the time.

How to read the calibration plot: - Diagonal line = perfect calibration - Points above the line = model under-predicts (too conservative) - Points below the line = model over-predicts (too confident)

Important interpretation guidance: Where you should focus depends on your outcome prevalence (shown in Setup): - **If your positive rate is low (< 30%)**: Most predictions will fall in the lower probability range (roughly 0 to 2-3× your positive rate). Focus your interpretation there. - **If your positive rate is balanced (30-70%)**: Predictions will be more spread out across the probability range. Examine calibration across the full range. - **If your positive rate is high (> 70%)**: Most predictions will fall in the higher probability range. Focus your interpretation there.

General caution: Bins with very few observations (typically n < 20) will show high variability due to small sample sizes and should be interpreted cautiously. These bins often appear at the extremes of the probability range.

```
# Create calibration plot with 5 bins
# Using fewer bins provides more stable estimates for imbalanced outcomes
calibration_plot(
  predProbs,
  ".pred_yes",
  learnerSpec$outcomeName,
  "learnerName",
  n_bins = 5
)
```



Question 5: Examine the calibration plot, focusing primarily on the probability range where most of your predictions fall (based on your outcome prevalence from Setup, this should be the lower range if you have a rare outcome, the full range if balanced, or the higher range if your outcome is common). Are your models reasonably well-calibrated in this relevant range (points close to the diagonal)? Which model appears best calibrated? Do any models show systematic over-prediction (points below diagonal) or under-prediction (points above diagonal)? Why should you be cautious about interpreting bins with very few observations, and where do these sparse bins typically appear in your plot?

In the lower range, all of the models are noticeably above the baseline. This suggests that our model is under-predicting in these cases, which makes sense based on the context. Our data is imbalanced (12% success) so the models likely inherited an aversion to predicting positive cases. The bins with very few observations are the higher end of the range, because of the previously mentioned imbalance. So we should be careful about interpreting bins on the right hand side of the graph.

---

5. Classification performance at different thresholds

So far, we’ve evaluated predicted probabilities. Now we’ll convert probabilities to classifications (yes/no) using different thresholds and evaluate the results.

Understanding thresholds: - Lower threshold (e.g., 0.1): Classify more as positive → catch more true positives but get more false positives - Higher threshold (e.g., 0.5): Be more conservative → fewer false positives but miss more true positives - Your choice depends on the relative costs of false positives vs false negatives in your context

5.1 Select thresholds to test

Choose 2-3 thresholds based on your prediction context and distributions of the predicted likelihoods you observed in the plots above.

```
fixedThresholds <- c(0.1, 0.2, 0.4)

cat("Testing thresholds:", fixedThresholds, "\n")
```

```
## Testing thresholds: 0.1 0.2 0.4
```

```
cat("\nGuidance for threshold selection:\n")
```

```
##
## Guidance for threshold selection:
```

```
cat("- Lower thresholds: More sensitive, catches more cases\n")
```

```
## - Lower thresholds: More sensitive, catches more cases
```

```
cat("- Higher thresholds: More specific, reduces false alarms\n")
```

```
## - Higher thresholds: More specific, reduces false alarms
```

```
cat("- Consider your outcome prevalence:",
    round(learnerSpec$positiveProportion, 3), "\n")
```

```
## - Consider your outcome prevalence: 0.117
```

Question 6: Why did you choose these specific threshold values? Explain how each threshold aligns with different priorities (e.g., maximizing a particular metric, or matching resource constraints). Given your prediction context and outcome prevalence, which threshold range seems most appropriate and why?

I selected these thresholds because in the context of marketing, I think missing a potential customer (false negative) is much more costly to us than thinking an uninterested customer wants to open an account (false positive). In marketing, we are expecting a ton of phone calls with no results. What can really hurt, is not calling someone that might have been a customer. Because of this, I chose to minimize false negatives by lowering the threshold values.

0.1 is designed to signal as many positive cases as possible, knowing that a lot of them will likely be false positive. 0.2 has basically the same goals, but is more balanced. I hypothesize this threshold will have the best balance between false positives and negatives in this context. And I chose 0.4 for a more balanced outlook, in this case I wanted to see how many less positive cases we end up with by increasing the threshold.

5.2 Key classification metrics

Understanding the metrics we'll calculate:

Rate metrics (expressed as proportions): - **TPR (True Positive Rate / Sensitivity / Recall)**: % of actual positives correctly identified - *higher is better* - **TNR (True Negative Rate / Specificity)**: % of actual negatives correctly identified - *higher is better* - **FPR (False Positive Rate)**: % of actual negatives incorrectly classified as positive - *lower is better* - **FNR (False Negative Rate)**: % of actual positives missed - *lower is better*

Other metrics: - **Precision (PPV)**: Of those predicted positive, what % are actually positive? - **F1 Score**: Harmonic mean of precision and recall (balances both) - **Accuracy**: Overall % of correct predictions (can be misleading with imbalanced data)

```
# Define metric groups for analysis
rateMetrics <- c("tpr", "fpr", "tnr", "fnr")
countMetrics <- c("tpc", "fpc", "tnc", "fnc")
otherMetrics <- c("ppv", "f1", "acc")
allMetrics <- c(rateMetrics, countMetrics, otherMetrics)
```

5.3 Calculate metrics for all learners

```
# Compute classification metrics at your specified thresholds
# This calculates all metrics for each learner at each threshold
learnerMetricsThres <- conf_metrics_thres_learner(
  predProbs = predProbs,
  outcomeName = learnerSpec$outcomeName,
  thresSeq = fixedThresholds
)

# Display metrics for easy comparison
cat("=== Classification Metrics by Learner and Threshold ===\n\n")
```

```
## === Classification Metrics by Learner and Threshold ===
```

```
# All Metrics
learnerMetricsThres %>%
  select(learnerName, threshold, all_of(allMetrics)) %>%
  arrange(learnerName, threshold) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()  # Show all rows
```

```
##                          learnerName threshold   tpr   fpr   tnr   fnr  tpc
## 1                     glm_predSet_bm       0.1 0.760 0.260 0.740 0.240 2812
## 2                     glm_predSet_bm       0.2 0.456 0.080 0.920 0.544 1689
## 3                     glm_predSet_bm       0.4 0.218 0.024 0.976 0.782  807
## 4              glm_predSet_extended1       0.1 0.801 0.276 0.724 0.199 2966
## 5              glm_predSet_extended1       0.2 0.553 0.106 0.894 0.447 2046
## 6              glm_predSet_extended1       0.4 0.279 0.032 0.968 0.721 1033
## 7              glm_predSet_extended2       0.1 0.788 0.272 0.728 0.212 2918
## 8              glm_predSet_extended2       0.2 0.473 0.084 0.916 0.527 1752
## 9              glm_predSet_extended2       0.4 0.224 0.026 0.974 0.776  831
## 10           glm_predSet_kitchensink       0.1 0.861 0.203 0.797 0.139 3186
## 11           glm_predSet_kitchensink       0.2 0.694 0.101 0.899 0.306 2571
## 12           glm_predSet_kitchensink       0.4 0.430 0.036 0.964 0.570 1593
## 13                  lasso_predSet_bm       0.1 1.000 1.000 0.000 0.000 3702
## 14                  lasso_predSet_bm       0.2 0.000 0.000 1.000 1.000    0
## 15                  lasso_predSet_bm       0.4 0.000 0.000 1.000 1.000    0
## 16           lasso_predSet_extended1       0.1 1.000 1.000 0.000 0.000 3702
## 17           lasso_predSet_extended1       0.2 0.000 0.000 1.000 1.000    0
## 18           lasso_predSet_extended1       0.4 0.000 0.000 1.000 1.000    0
## 19           lasso_predSet_extended2       0.1 1.000 1.000 0.000 0.000 3702
## 20           lasso_predSet_extended2       0.2 0.000 0.000 1.000 1.000    0
## 21           lasso_predSet_extended2       0.4 0.000 0.000 1.000 1.000    0
## 22         lasso_predSet_kitchensink       0.1 1.000 1.000 0.000 0.000 3702
## 23         lasso_predSet_kitchensink       0.2 0.000 0.000 1.000 1.000    0
## 24         lasso_predSet_kitchensink       0.4 0.000 0.000 1.000 1.000    0
## 25   random_forest_predSet_extended1       0.1 0.856 0.277 0.723 0.144 3170
## 26   random_forest_predSet_extended1       0.2 0.672 0.131 0.869 0.328 2486
## 27   random_forest_predSet_extended1       0.4 0.329 0.036 0.964 0.671 1218
## 28   random_forest_predSet_extended2       0.1 0.817 0.250 0.750 0.183 3024
## 29   random_forest_predSet_extended2       0.2 0.667 0.129 0.871 0.333 2469
## 30   random_forest_predSet_extended2       0.4 0.337 0.039 0.961 0.663 1248
## 31 random_forest_predSet_kitchensink       0.1 0.928 0.238 0.762 0.072 3434
## 32 random_forest_predSet_kitchensink       0.2 0.824 0.130 0.870 0.176 3051
## 33 random_forest_predSet_kitchensink       0.4 0.510 0.044 0.956 0.490 1887
## 34         xgboost_predSet_extended1       0.1 0.880 0.283 0.717 0.120 3257
## 35         xgboost_predSet_extended1       0.2 0.694 0.138 0.862 0.306 2568
## 36         xgboost_predSet_extended1       0.4 0.382 0.044 0.956 0.618 1414
## 37         xgboost_predSet_extended2       0.1 0.832 0.261 0.739 0.168 3080
## 38         xgboost_predSet_extended2       0.2 0.679 0.134 0.866 0.321 2515
## 39         xgboost_predSet_extended2       0.4 0.360 0.042 0.958 0.640 1334
## 40       xgboost_predSet_kitchensink       0.1 0.900 0.219 0.781 0.100 3333
## 41       xgboost_predSet_kitchensink       0.2 0.763 0.112 0.888 0.237 2825
## 42       xgboost_predSet_kitchensink       0.4 0.498 0.044 0.956 0.502 1843
##      fpc   tnc  fnc   ppv    f1   acc
## 1   7259 20686  890 0.279 0.408 0.743
## 2   2247 25698 2013 0.429 0.442 0.865
## 3    678 27267 2895 0.543 0.311 0.887
## 4   7703 20242  736 0.278 0.413 0.733
## 5   2957 24988 1656 0.409 0.470 0.854
## 6    898 27047 2669 0.535 0.367 0.887
## 7   7601 20344  784 0.277 0.410 0.735
## 8   2358 25587 1950 0.426 0.449 0.864
```

```
## 9    713 27232 2871 0.538 0.317 0.887
## 10  5680 22265  516 0.359 0.507 0.804
## 11  2827 25118 1131 0.476 0.565 0.875
## 12   997 26948 2109 0.615 0.506 0.902
## 13 27945     0    0 0.117 0.209 0.117
## 14     0 27945 3702   NaN   NaN 0.883
## 15     0 27945 3702   NaN   NaN 0.883
## 16 27945     0    0 0.117 0.209 0.117
## 17     0 27945 3702   NaN   NaN 0.883
## 18     0 27945 3702   NaN   NaN 0.883
## 19 27945     0    0 0.117 0.209 0.117
## 20     0 27945 3702   NaN   NaN 0.883
## 21     0 27945 3702   NaN   NaN 0.883
## 22 27945     0    0 0.117 0.209 0.117
## 23     0 27945 3702   NaN   NaN 0.883
## 24     0 27945 3702   NaN   NaN 0.883
## 25  7729 20216  532 0.291 0.434 0.739
## 26  3673 24272 1216 0.404 0.504 0.846
## 27  1004 26941 2484 0.548 0.411 0.890
## 28  6973 20972  678 0.302 0.441 0.758
## 29  3592 24353 1233 0.407 0.506 0.848
## 30  1077 26868 2454 0.537 0.414 0.888
## 31  6663 21282  268 0.340 0.498 0.781
## 32  3632 24313  651 0.457 0.588 0.865
## 33  1235 26710 1815 0.604 0.553 0.904
## 34  7910 20035  445 0.292 0.438 0.736
## 35  3858 24087 1134 0.400 0.507 0.842
## 36  1217 26728 2288 0.537 0.447 0.889
## 37  7298 20647  622 0.297 0.438 0.750
## 38  3757 24188 1187 0.401 0.504 0.844
## 39  1186 26759 2368 0.529 0.429 0.888
## 40  6113 21832  369 0.353 0.507 0.795
## 41  3119 24826  877 0.475 0.586 0.874
## 42  1223 26722 1859 0.601 0.545 0.903
```

```
# Competitive Learners
learnerMetricsThres %>%
  filter(learnerName %in% c("random_forest_predSet_kitchensink", "xgboost_predSet_kitchensink",
"glm_predSet_kitchensink"),
         threshold %in% c(0.1, 0.2, 0.4)) %>%
  select(learnerName, threshold, tpr, fpr, fnr, ppv, f1) %>%
  arrange(desc(f1)) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
##                         learnerName threshold   tpr   fpr   fnr   ppv    f1
## 1 random_forest_predSet_kitchensink       0.2 0.824 0.130 0.176 0.457 0.588
## 2       xgboost_predSet_kitchensink       0.2 0.763 0.112 0.237 0.475 0.586
## 3           glm_predSet_kitchensink       0.2 0.694 0.101 0.306 0.476 0.565
## 4 random_forest_predSet_kitchensink       0.4 0.510 0.044 0.490 0.604 0.553
## 5       xgboost_predSet_kitchensink       0.4 0.498 0.044 0.502 0.601 0.545
## 6           glm_predSet_kitchensink       0.1 0.861 0.203 0.139 0.359 0.507
## 7       xgboost_predSet_kitchensink       0.1 0.900 0.219 0.100 0.353 0.507
## 8           glm_predSet_kitchensink       0.4 0.430 0.036 0.570 0.615 0.506
## 9 random_forest_predSet_kitchensink       0.1 0.928 0.238 0.072 0.340 0.498
```

Question 9: After reviewing the full metrics table, create a custom filtered view (using the template above or your own code) that helps you compare the learners and thresholds you're most interested in. Focus on the metrics most relevant to your prediction context. Based on this filtered view, what patterns or insights emerge? Which learner-threshold combinations look most promising for the predictive analytics use case that you scoped?

Parallel to the logic earlier, I am putting an emphasis on false negative rate. I want to minimize false negative rate so that we call everybody who would end up opening an account. Leaving customers un-contacted when they would have opened an account would be costly for the bank. As a result, this is one of the metrics I chose. I also wanted to look at TPR, FPR, precision, and F1 score. With an emphasis on FNR, and the rest of the values for context, I think we can compare/evaluate our models effectively. After looking at the metrics, I think the threshold of 0.1 most accurately addresses what we want our model to do.

We are trying to predict for all customers, at all points in time during the marketing campaign. So the raw metrics are very telling when we think about our scoping, and minimizing false negatives is in my opinion the best suggestion we can make to the bank.

5.5 Visualize confusion matrix statistics

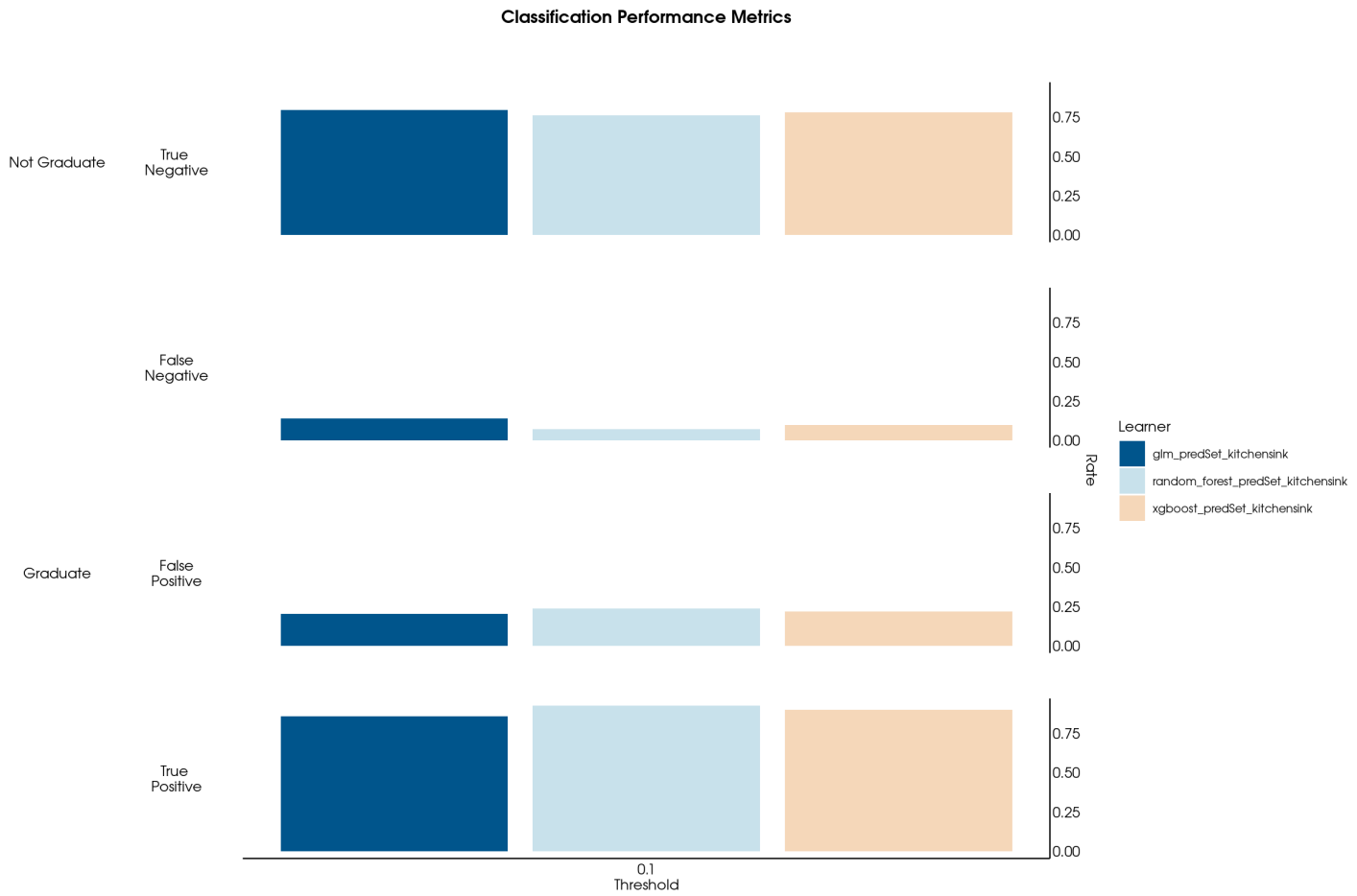
Based on the results above, you may want to narrow down to the most promising learners and thresholds for closer examination. These bar plots provide a visual comparison of classification performance, making it easier to see patterns and trade-offs.

What the plots show: - Each panel displays one of the four key rate metrics (TPR, FPR, TNR, FNR) - Or you can chose to display the count metrics (TPC, FPC, TNC, FNC) - Each bar represents one learner at one threshold - Bars are grouped by threshold to facilitate comparison

How to interpret: - **TPR/TPC and TNR/TNC**: Higher is better (bars closer to 1.0 indicate better performance) - **FPR/FPC and FNR/FNC**: Lower is better (bars closer to 0.0 indicate fewer errors) - Compare bars within each panel to identify which learners perform best on that metric - Compare the same learner across thresholds to see how changing the threshold affects performance

Key insight: There's always a trade-off between metrics. A learner with high TPR (catching most positive cases) will typically have higher FPR (more false alarms). Your goal is to find the best balance for your specific prediction context/scoping.

```r
selectedLearners <- c("glm_predSet_kitchensink", "random_forest_predSet_kitchensink", "xgboost_p
redSet_kitchensink")

selectedThresholds <- c(0.1, 0.15)

selectMetricsGroup <- rateMetrics

# Narrow results to selections
# No more edits are needed from here
# learnerMetricsThres %>%
#   filter(threshold %in% selectedThresholds) %>%
#   filter(learnerName %in% selectedLearners) %>%
#   select(learnerName, all_of(selectMetricsGroup))

# Plot
plotData <- learnerMetricsThres %>%
  filter(learnerName %in% selectedLearners) %>%
  filter(threshold %in% selectedThresholds) %>%
  pivot_longer(
    cols = all_of(allMetrics),
    names_to = "metricType",
    values_to = "value"
  )

conf_mat_bar_plot(
   plotData = plotData %>% filter(metricType %in% rateMetrics),
   metricType = "rate",
   outcomeDescription = c("Not Graduate", "Graduate"),
   graphTitle = "Classification Performance Metrics"
)
```



Question 10: Examine the bar plots carefully. Which learner-threshold combination achieves the best balance between TPR (catching true positives) and FPR (avoiding false positives) for your prediction context? Looking across thresholds, describe how increasing the threshold from your lowest to highest value affects all four rate metrics. Does the visualization reveal any patterns or insights that weren't immediately obvious from the tables alone?

The model that best balances TPR and FPR in this context is random_forest_predSet_kitchensink. It has the highest TPR, which is the most important when trying to identify a small proportion of positive cases, and also manages false positives very close to as well as the other two learners. When you increase the threshold, FNR increases, but FPR decreases. Also, TPR decreases, while TNR increases. There are tradeoffs to your threshold, so it is important to understand the problem you are trying to solve and your scope when choosing.

---

6. Fairness assessment

Assess whether your model performs differently across groups. Fairness is crucial for ethical deployment of predictive models.

What we're checking: - Do predicted probabilities differ across groups? - Does the model have similar performance across groups? - Are there biases that could lead to unfair outcomes if the model was deployed? - Do different learners have different results related to bias?

6.1 Distribution of predictions by group

The following code only allows you to look at one learner at time. At this point you may have narrowed your learners to one or two based on performance. You may want to examine bias for more than one learner. Therefore, you will need to copy and paste/edit the code below and repeat it for another learner to get plots for each.

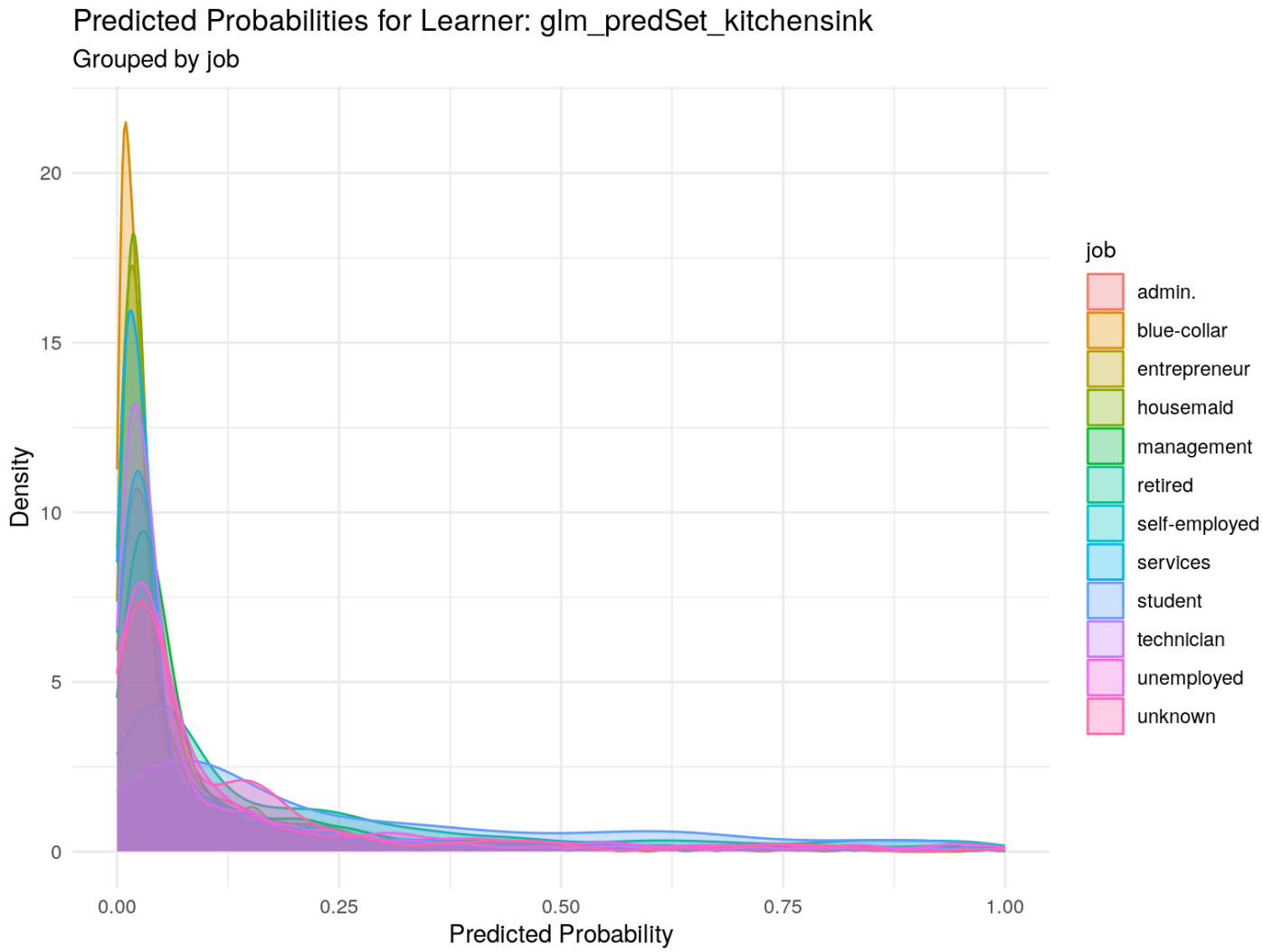
```
fairnessLearner <- "glm_predSet_kitchensink"
fairnessLearner2 <- "random_forest_predSet_kitchensink"
fairnessLearner3 <- "xgboost_predSet_kitchensink"

# Get the equity variable from your Milestone 4 specifications
equityVar <- learnerSpec$equityVars[1]

cat("Assessing fairness for:", fairnessLearner, "\n")
```

```
## Assessing fairness for: glm_predSet_kitchensink
```

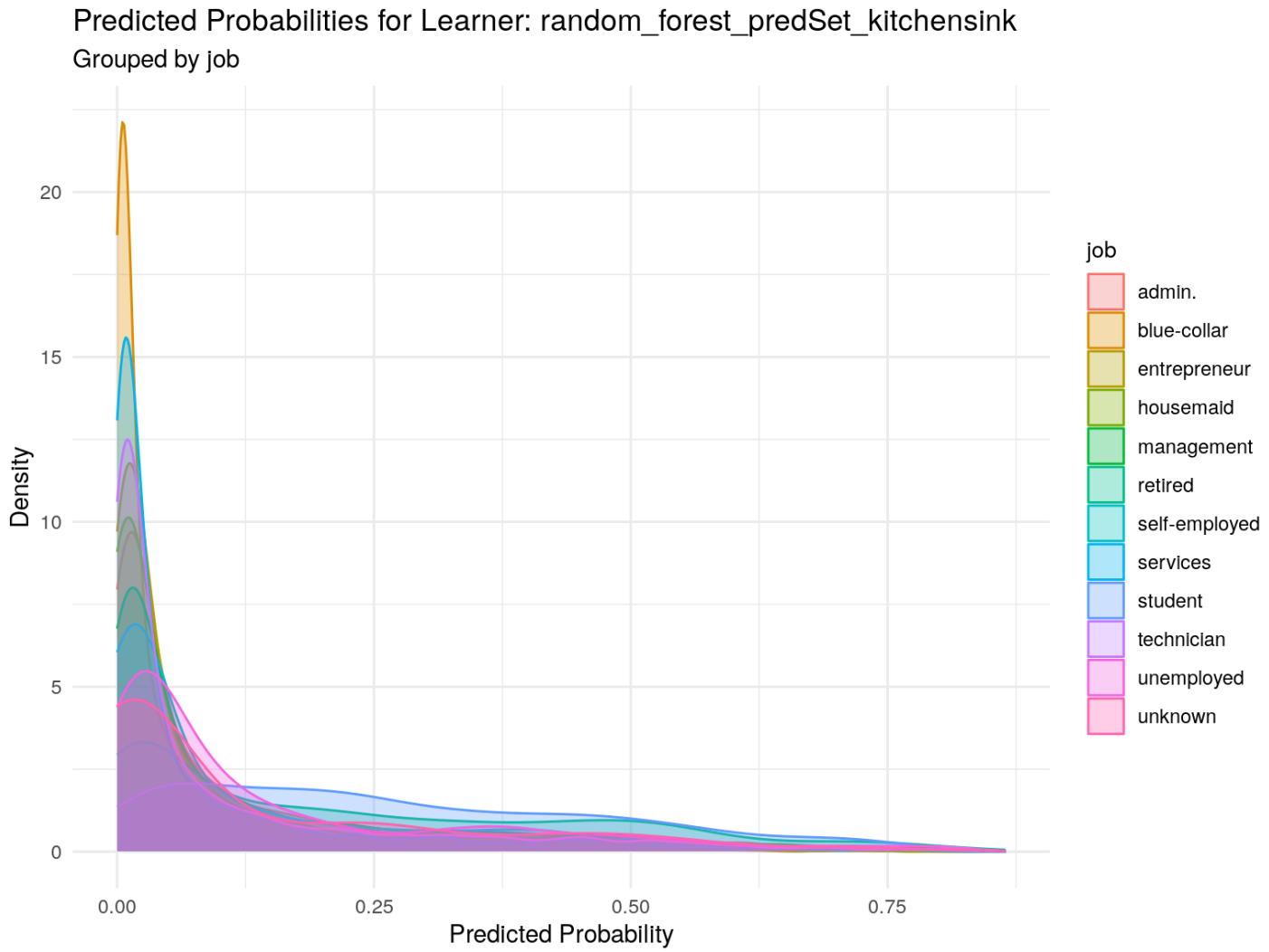
```
cat("Equity variable:", equityVar, "\n\n")
```

```
## Equity variable: job
```

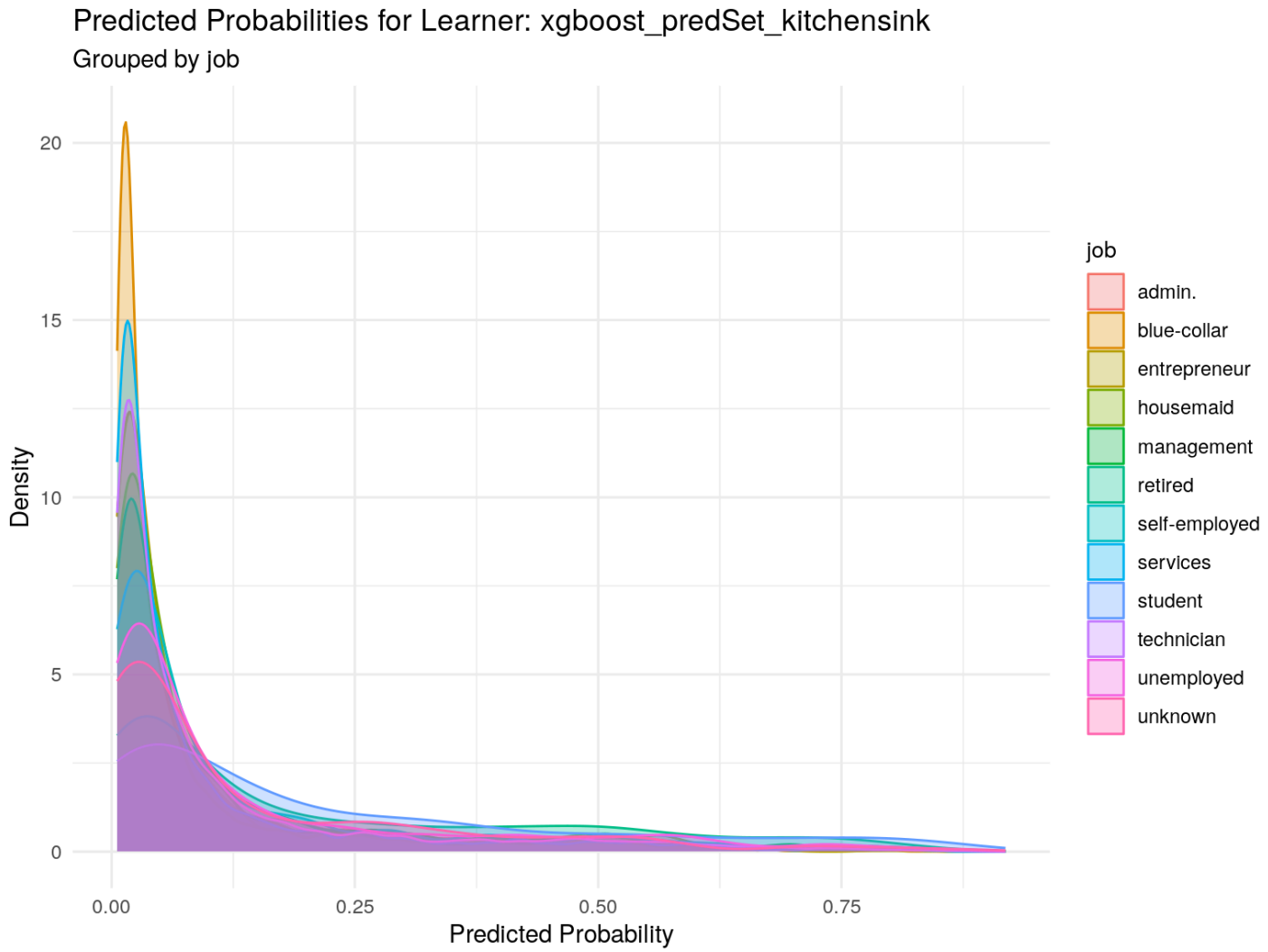
```
# Plot distribution of predicted probabilities by group
plot_pred_probs(
  predProbs,
  learner = fairnessLearner,
  group_var = equityVar
)
```




```
# Plot distribution of predicted probabilities by group
plot_pred_probs(
  predProbs,
  learner = fairnessLearner2,
  group_var = equityVar
)
```




```
# Plot distribution of predicted probabilities by group
plot_pred_probs(
  predProbs,
  learner = fairnessLearner3,
  group_var = equityVar
)
```



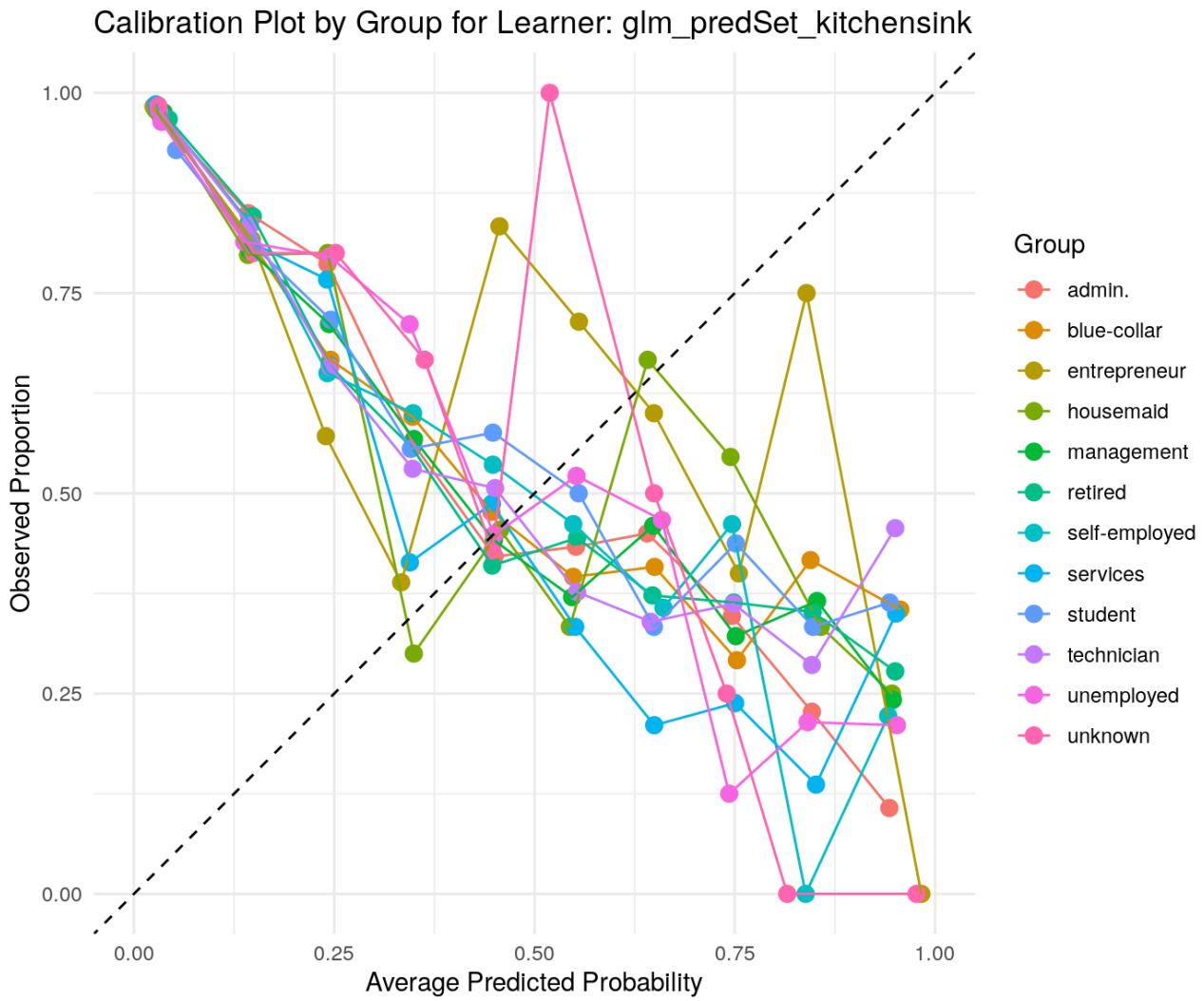
Question 11: Examine the distribution of predicted probabilities across groups. Do you observe differences in where the distributions are centered or in their spread? Which group(s) tend to receive higher predicted probabilities? What might explain these differences - could they reflect true differences in outcome rates between groups, differences in predictor availability/quality, or potential model bias? What else could they reflect?

The distributions all seem to be centered around the same point, and have similar spreads. All of the groups have low predicted probabilities. Because our data does not inherently have ethnic/racial/gender data, it kind of makes sense that we wouldn't notice any problems with fairness. It seems like our models are unbiased.

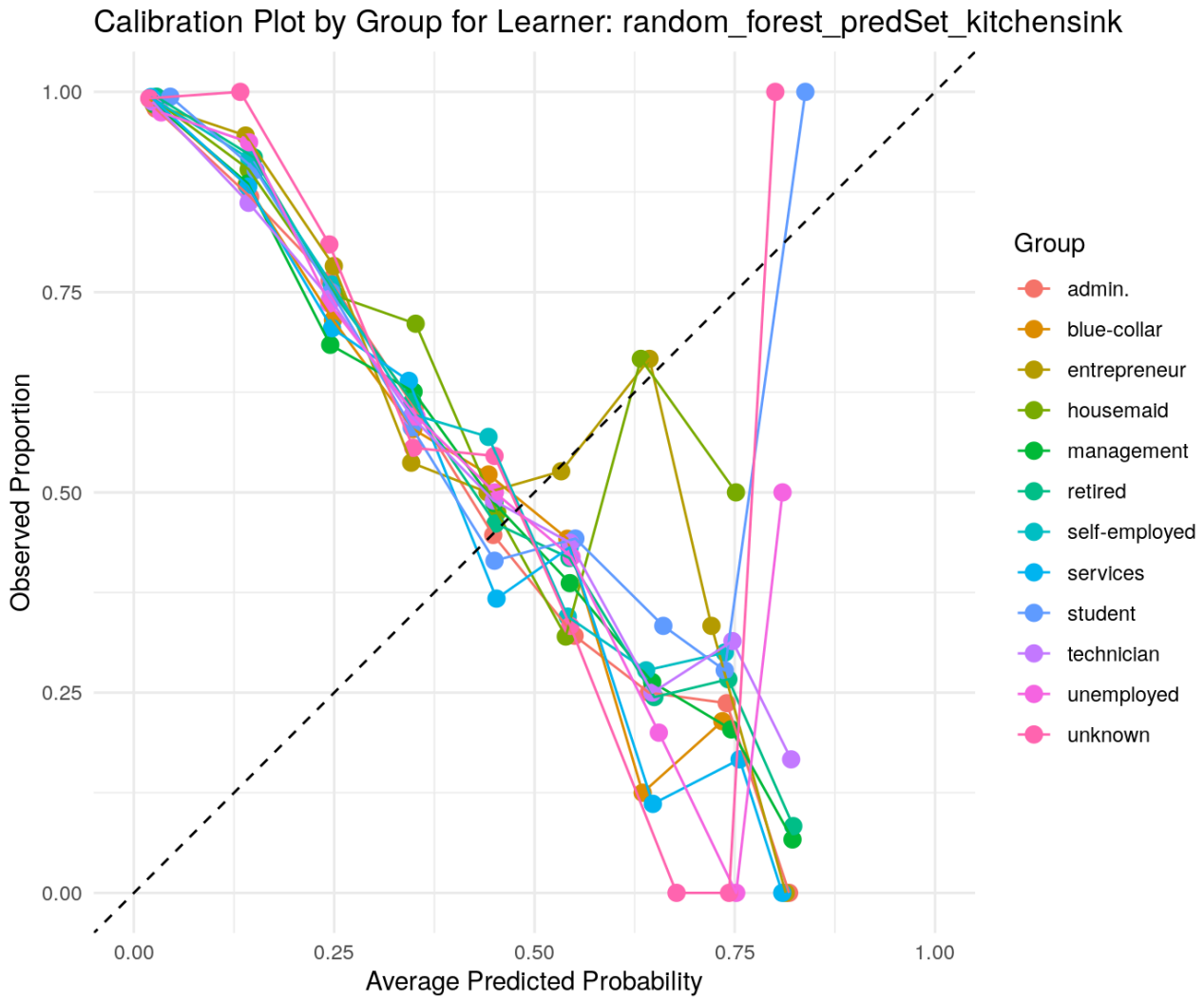
6.2 Calibration by group

Check whether the model is equally well-calibrated across demographic groups. Again, you can copy and paste this code and change what you pass in for learnerName to examine calibration for another learner.

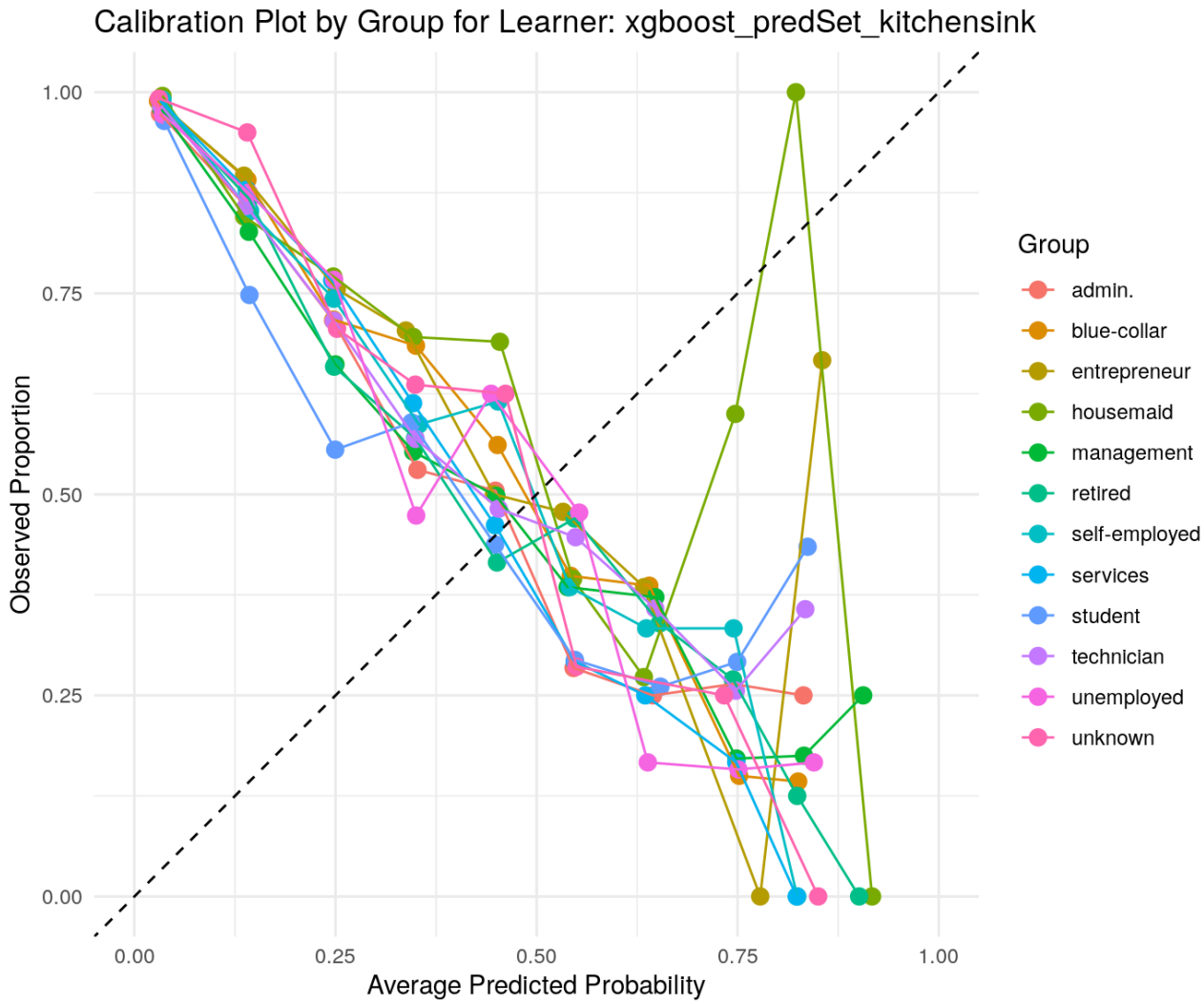
```
# Plot calibration separately for each demographic group
calibration_plot_by_group(
  predProbs,
  ".pred_yes",
  learnerSpec$outcomeName,
  group_col = equityVar,
  learner_col = "learnerName",
  learnerName = fairnessLearner
)
```




```
# Plot calibration separately for each demographic group
calibration_plot_by_group(
  predProbs,
  ".pred_yes",
  learnerSpec$outcomeName,
  group_col = equityVar,
  learner_col = "learnerName",
  learnerName = fairnessLearner2
)
```




```
# Plot calibration separately for each demographic group
calibration_plot_by_group(
  predProbs,
  ".pred_yes",
  learnerSpec$outcomeName,
  group_col = equityVar,
  learner_col = "learnerName",
  learnerName = fairnessLearner3
)
```



Question 12: Compare calibration across demographic groups. Is the model equally well-calibrated for all groups (points equally close to the diagonal line)? Are there groups for which the model systematically over-predicts (points below diagonal) or under-predicts (points above diagonal)? What are the practical implications if a model is poorly calibrated for certain groups?

There are a few groups that seem to have some interesting behavior. In the random forest model (current favorite), we can see students and unemployed jump above on the right hand side. This shows underfitting in these areas, but it is important to note that we have very few observations on that right hand side because of our data imbalance.

There is also some variability across models, in terms of which groups deviate/stick to the baseline. Similarly to the last calibration graph though, the data starts in the upper left, above the baseline, and eventually dips below as we move in the positive direction along the x-axis.

In real life, poor calibration can have negative impacts. In this case, that could look like reaching out a ton to an uninterested group, or not contacting a group that has genuine interest.

6.3 Classification metrics by group

Examine whether classification accuracy differs across groups at your chosen threshold.

```
fairnessThreshold <- 0.25

cat("=== Fairness Assessment ===\n")
```

```
## === Fairness Assessment ===
```

```
cat("Learner:", fairnessLearner, "\n")
```

```
## Learner: glm_predSet_kitchensink
```

```
cat("Threshold:", fairnessThreshold, "\n")
```

```
## Threshold: 0.25
```

```
cat("Equity variable:", equityVar, "\n\n")
```

```
## Equity variable: job
```

```
# Calculate metrics by group
equityResults <- calc_equity_metrics_all(
  predProbs = predProbs,
  learnerNames = c(fairnessLearner, fairnessLearner2, fairnessLearner3),
  thresholdsSelect = fairnessThreshold,
  equityVars = equityVar,
  outcomeName = learnerSpec$outcomeName
)

# Display key metrics by group
cat("Classification metrics by group:\n\n")
```

```
## Classification metrics by group:
```

```
equityResults %>%
  select(category, threshold, tpr, fpr, ppv, fnr) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
##         category threshold   tpr   fpr   ppv   fnr
## 1         admin.      0.25 0.617 0.077 0.528 0.383
## 2    blue-collar      0.25 0.476 0.040 0.479 0.524
## 3   entrepreneur      0.25 0.369 0.050 0.397 0.631
## 4      housemaid      0.25 0.519 0.047 0.512 0.481
## 5     management      0.25 0.655 0.095 0.537 0.345
## 6        retired      0.25 0.737 0.176 0.545 0.263
## 7  self-employed      0.25 0.632 0.076 0.525 0.368
## 8       services      0.25 0.582 0.041 0.579 0.418
## 9        student      0.25 0.732 0.265 0.512 0.268
## 10    technician      0.25 0.573 0.063 0.522 0.427
## 11    unemployed      0.25 0.667 0.115 0.508 0.333
## 12       unknown      0.25 0.542 0.065 0.542 0.458
## 13        admin.      0.25 0.689 0.088 0.524 0.311
## 14   blue-collar      0.25 0.712 0.062 0.469 0.288
## 15  entrepreneur      0.25 0.714 0.084 0.432 0.286
## 16     housemaid      0.25 0.734 0.098 0.417 0.266
## 17    management      0.25 0.762 0.120 0.519 0.238
## 18       retired      0.25 0.883 0.230 0.525 0.117
## 19 self-employed      0.25 0.820 0.121 0.474 0.180
## 20      services      0.25 0.737 0.062 0.535 0.263
## 21       student      0.25 0.844 0.314 0.505 0.156
## 22    technician      0.25 0.714 0.092 0.483 0.286
## 23    unemployed      0.25 0.790 0.131 0.517 0.210
## 24       unknown      0.25 0.875 0.124 0.500 0.125
## 25        admin.      0.25 0.644 0.075 0.546 0.356
## 26   blue-collar      0.25 0.682 0.065 0.448 0.318
## 27  entrepreneur      0.25 0.679 0.080 0.432 0.321
## 28     housemaid      0.25 0.684 0.093 0.412 0.316
## 29    management      0.25 0.683 0.099 0.539 0.317
## 30       retired      0.25 0.826 0.189 0.556 0.174
## 31 self-employed      0.25 0.744 0.101 0.495 0.256
## 32      services      0.25 0.721 0.062 0.528 0.279
## 33       student      0.25 0.704 0.197 0.575 0.296
## 34    technician      0.25 0.670 0.080 0.499 0.330
## 35    unemployed      0.25 0.732 0.115 0.532 0.268
## 36       unknown      0.25 0.833 0.135 0.465 0.167
```

Question 13: Examine the classification metrics across demographic groups carefully. Do you see meaningful differences in TPR (sensitivity) across groups? What about FPR (false positive rate)? Or other key metrics that you care about for your scope? Consider the context of your scope: if one group has a much higher FPR, what does this mean in practice for members of that group? If one group has a lower TPR, what opportunities might they be missing? Which disparities concern you most from an ethical standpoint and why? What factors might explain these differences (sample size, base rates, feature availability, or actual bias)?

There do seem to be significant differences across groups in both TPR and FPR. In context, this could negatively effect how the bank manages it's resources. An abnormally high FPR can lead to the bank over-committing to groups that are not interested, and a low TPR could lead to an interested group not getting adequately targeted. This can make a very big difference in business.

Because of the lack of sensitive information and personal data, I am not too concerned ethically about the disparities. But from a business perspective, they could certainly have an impact. Overall, the data is fair, but it is always important to look into these things throughout a project cycle.

---

7. Results summary

Before selecting your final model, review this comprehensive summary of all results.

```
cat("============================================\n")
```

```
## ============================================
```

```
cat("         COMPLETE RESULTS SUMMARY\n")
```

```
##          COMPLETE RESULTS SUMMARY
```

```
cat("============================================\n\n")
```

```
## ============================================
```

```
# 1. Discrimination performance
cat("1. DISCRIMINATION (Cross-Validated)\n")
```

```
## 1. DISCRIMINATION (Cross-Validated)
```

```
cat("   Sorted by PR-AUC (most important for imbalanced outcomes)\n\n")
```

```
##    Sorted by PR-AUC (most important for imbalanced outcomes)
```

```
modelResults %>%
  filter(.metric == "pr_auc") %>%
  select(learnerName, mean, std_err) %>%
  arrange(desc(mean)) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
## # A tibble: 14 × 3
##    learnerName                        mean std_err
##    <chr>                             <dbl>   <dbl>
##  1 random_forest_predSet_kitchensink 0.592   0.012
##  2 xgboost_predSet_kitchensink       0.579   0.012
##  3 lasso_predSet_bm                  0.558   0
##  4 lasso_predSet_extended1           0.558   0
##  5 lasso_predSet_extended2           0.558   0
##  6 lasso_predSet_kitchensink         0.558   0
##  7 glm_predSet_kitchensink           0.545   0.018
##  8 xgboost_predSet_extended1         0.464   0.007
##  9 random_forest_predSet_extended1   0.451   0.007
## 10 xgboost_predSet_extended2         0.448   0.006
## 11 random_forest_predSet_extended2   0.442   0.006
## 12 glm_predSet_extended1             0.423   0.01
## 13 glm_predSet_extended2             0.405   0.011
## 14 glm_predSet_bm                    0.398   0.01
```

```
# 2. Classification metrics at focus threshold
focusThreshold <- 0.1

cat("\n2. CLASSIFICATION METRICS AT THRESHOLD", focusThreshold, "\n\n")
```

```
##
## 2. CLASSIFICATION METRICS AT THRESHOLD 0.1
```

```
learnerMetricsThres %>%
  filter(threshold == focusThreshold) %>%
  select(learnerName, tpr, fpr, fnr, ppv, f1) %>%
  arrange(desc(f1)) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
##                          learnerName   tpr   fpr   fnr   ppv    f1
## 1            glm_predSet_kitchensink 0.861 0.203 0.139 0.359 0.507
## 2        xgboost_predSet_kitchensink 0.900 0.219 0.100 0.353 0.507
## 3  random_forest_predSet_kitchensink 0.928 0.238 0.072 0.340 0.498
## 4    random_forest_predSet_extended2 0.817 0.250 0.183 0.302 0.441
## 5          xgboost_predSet_extended1 0.880 0.283 0.120 0.292 0.438
## 6          xgboost_predSet_extended2 0.832 0.261 0.168 0.297 0.438
## 7    random_forest_predSet_extended1 0.856 0.277 0.144 0.291 0.434
## 8              glm_predSet_extended1 0.801 0.276 0.199 0.278 0.413
## 9              glm_predSet_extended2 0.788 0.272 0.212 0.277 0.410
## 10                    glm_predSet_bm 0.760 0.260 0.240 0.279 0.408
## 11                  lasso_predSet_bm 1.000 1.000 0.000 0.117 0.209
## 12           lasso_predSet_extended1 1.000 1.000 0.000 0.117 0.209
## 13           lasso_predSet_extended2 1.000 1.000 0.000 0.117 0.209
## 14         lasso_predSet_kitchensink 1.000 1.000 0.000 0.117 0.209
```

```r
# 3. Fairness summary (if applicable)
if (exists("equityResults")) {
  cat("\n3. FAIRNESS SUMMARY\n")
  cat("   TPR by group at threshold", fairnessThreshold, "\n\n")
  equityResults %>%
    select(category, tpr, fpr) %>%
    mutate(across(where(is.numeric), ~round(., 3))) %>%
    print()
}
```

```
##
## 3. FAIRNESS SUMMARY
##    TPR by group at threshold 0.25
##
##         category   tpr   fpr
## 1         admin. 0.617 0.077
## 2    blue-collar 0.476 0.040
## 3   entrepreneur 0.369 0.050
## 4      housemaid 0.519 0.047
## 5     management 0.655 0.095
## 6        retired 0.737 0.176
## 7  self-employed 0.632 0.076
## 8       services 0.582 0.041
## 9        student 0.732 0.265
## 10    technician 0.573 0.063
## 11    unemployed 0.667 0.115
## 12       unknown 0.542 0.065
## 13        admin. 0.689 0.088
## 14   blue-collar 0.712 0.062
## 15  entrepreneur 0.714 0.084
## 16     housemaid 0.734 0.098
## 17    management 0.762 0.120
## 18       retired 0.883 0.230
## 19 self-employed 0.820 0.121
## 20      services 0.737 0.062
## 21       student 0.844 0.314
## 22    technician 0.714 0.092
## 23    unemployed 0.790 0.131
## 24       unknown 0.875 0.124
## 25        admin. 0.644 0.075
## 26   blue-collar 0.682 0.065
## 27  entrepreneur 0.679 0.080
## 28     housemaid 0.684 0.093
## 29    management 0.683 0.099
## 30       retired 0.826 0.189
## 31 self-employed 0.744 0.101
## 32      services 0.721 0.062
## 33       student 0.704 0.197
## 34    technician 0.670 0.080
## 35    unemployed 0.732 0.115
## 36       unknown 0.833 0.135
```

```
cat("\n==========================================\n")
```

```
##
## ==========================================
```

---

8. Final learner selection

Now make your final decision using this structured framework.

Step 1: Narrow down candidates

Based on your analysis, eliminate poor performers and identify top candidates.

Criteria to consider: - Discrimination: PR-AUC and ROC-AUC - Calibration: Based on calibration plot - Classification: Confusion matrix metrics - Fairness: Differences in performanc across groups - Interpretability: Simpler models (glm, lasso) are easier to explain

Step 2: Consider simplicity and tranparency

Question 14: How important is model interpretability in your prediction context? Would stakeholders (decision-makers, affected individuals, regulators) need to understand why specific predictions are made? If interpretability is important, how should this influence your choice between simpler models (like glm) versus more complex “black box” models (like random forest or xgboost)? What would you lose and gain by prioritizing interpretability?

I think intrerpretability is important in bank marketing, but I also think that the financial industry has come to an understanding that data is valuable. I think stakeholders have placed more and more trust in data scientists/analysts in the recent years, especially in banking. That being said, it is always important to be able to explain your decisions to non-technical audiences.

Originally, I thought the regression and random forest models were going to be close enough in performance that I would opt for the GLM. Through this comparison process though, I have realized that the random forest model has statistically significant improvements over the GLM model, and is still simple enough to be able to explain (compared to really complex ML algorithms). In this case, I would lose positive cases, (lower TPR, higher FNR in GLM) and thus potential customers by opting for the GLM model.

As a result, I am choosing the random forest model with the kitchensink predictor set.

Step 3: Make your selection

Select both a learner and the threshold you would recommend for turning predicted likelihoods into classifications for your scoped use-case.

```
finalLearner <- "random_forest_predSet_kitchensink"

finalThresholds <- c(0.1, 0.15)

cat("==========================================\n")
```

```
## ============================================
```

```
cat("         FINAL SELECTION\n")
```

```
##          FINAL SELECTION
```

```
cat("============================================\n\n")
```

```
## ============================================
```

```
cat("Selected learner:", finalLearner, "\n")
```

```
## Selected learner: random_forest_predSet_kitchensink
```

```
cat("Selected threshold(s):", finalThresholds, "\n\n")
```

```
## Selected threshold(s): 0.1 0.15
```

```
# Display final learner's performance
cat("Discrimination:\n")
```

```
## Discrimination:
```

```
modelResults %>%
  filter(learnerName == finalLearner) %>%
  select(.metric, mean, std_err) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
## # A tibble: 2 × 3
##   .metric  mean std_err
##   <chr>   <dbl>   <dbl>
## 1 pr_auc  0.592   0.012
## 2 roc_auc 0.922   0.002
```

```
cat("\nClassification metrics at selected threshold(s):\n")
```

```
##
## Classification metrics at selected threshold(s):
```

```
learnerMetricsThres %>%
  filter(learnerName == finalLearner, threshold %in% finalThresholds) %>%
  select(threshold, all_of(allMetrics)) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
##   threshold   tpr   fpr   tnr   fnr  tpc  fpc   tnc fnc  ppv    f1   acc
## 1       0.1 0.928 0.238 0.762 0.072 3434 6663 21282 268 0.34 0.498 0.781
```

```
cat("\n===========================================\n")
```

```
##
## ===========================================
```

Step 4: Justify your choice

Question 15: Provide a comprehensive justification for your final learner and threshold selection. Your response must address ALL of the following:

1. **Performance justification**: Why this learner over others? Cite specific metrics (PR-AUC, calibration, TPR/FPR at your threshold) that support your choice.
2. **Threshold justification**: Why this specific threshold? Explain how it balances the trade-offs between sensitivity and precision given the relative costs of errors in your context.
3. **Context alignment**: How does this choice align with your prediction objective, population, and timepoint (your scope)?
4. **Fairness consideration**: How did fairness results influence your decision? If disparities exist, justify why you're comfortable proceeding or explain what mitigation steps you'd recommend.
5. **Limitations and trade-offs**: What are you giving up with this choice? What are the key limitations or risks you need to be aware of when recommending this model moves forward for testing?

I am choosing the random forest model for a few key reasons. The first is that it was the best with discrimination and separation. This is important in our context. On top of that, it had a very strong ROC-AUC, and its PR-AUC, which we value more, was very comparable with other contending models. What really drove the decision was when we got to the metrics. The random forest was able to achieve the highest TPR, and lowest FNR, which I am putting emphasis on becuase of the imbalance in our data.

I chose the thresholds of (0.1, 0.15) because I wanted to minimize FNR. As a result, we are going to go with a lower threshold. When I was computing the metrics, the thresholds (0.1, 0.2) were providing much more attractive numbers for this context than 0.4. I liked the numbers from 0.1 the best, but realized they were extreme, so I added the 0.15 threshold to give a slightly more balanced option.

Through the entire model building process, the scope of the project was at the front of mind. As a result, I think that my choices align with the scoping very well. I am trying to give the bank my most valuable suggestion and advice about how to market, and I built this model with that in mind.

The fairness results did not have too much of an impact on my decision making. The banking data is fair, and there is no race/gender/true demographic information in the dataset. As a result, our models were naturally quite fair. Any disparities that may have existed, likely exist because they are true and actually hold predictive power.

There are 2 main things I am giving up with this model. First, I am allowing the false positive rate to be higher than it could be. As a result, the model will predict positive cases, that the bank follows up on and does not get a sale. On top of this, I am giving up TNR. There will be negative outcomes that do not get properly identified, again as a result of how we made these decisions.

The philosophy is this: I would rather call and get told no, than not contact a likely client. There is a reason companies measure conversion rate, you cannot make a sale on every pitch. But in my opinion, it is better to pitch to customers that are unlikely to purchase on occasion, than to miss out on pitching clients that would have bought the product.

---

9. Save results for testing

Your selected learner will be tested on the held-out test set in the next milestone.

```
# Create object with selected learner results
trainResultsValidate <- filter_results(learnerResults, finalLearner)

# Add your selected thresholds
trainResultsValidate$selectedThresholds <- finalThresholds

# Save for next milestone
saveRDS(
  trainResultsValidate,
  file = paste0(params$my_path, "validateLearnerResults.rds")
)

cat("Results saved successfully!\n")
```

```
## Results saved successfully!
```

```
cat("File:", paste0(params$my_path, "validatelearnerResults.rds"), "\n")
```

```
## File: /share/dsa495601f25/tmgrover/validatelearnerResults.rds
```

```
# Clean up parallel processing
stopCluster(cl)
registerDoSEQ()
cat("\nParallel processing stopped.\n")
```

```
##
## Parallel processing stopped.
```

Appendix: Advanced threshold selection (Optional)

If you want to explore more sophisticated threshold selection methods, use this section.

A. Distribution-based thresholds

```
# Calculate median and proportional thresholds for each learner
distributionThresholds <- predProbs %>%
  group_by(learnerName) %>%
  summarise(
    median_threshold = median(.pred_yes),
    proportional_threshold = quantile(.pred_yes,
                                      probs = learnerSpec$positiveProportion)
  )

print(distributionThresholds)
```

B. Criteria-based thresholds

```
# Find threshold that achieves target TPR
criteriaThreshold <- find_threshold(
  predProbs,
  predicted_probs_col = ".pred_yes",
  ground_truths_col = learnerSpec$outcomeName,
  learner_col = "learnerName",
  metric = "tpr",
  target_proportion = 0.8  # Target 80% sensitivity
)

print(criteriaThreshold)
```

Congratulations! You have completed Milestone 5. Make sure you have: - [*] Answered all questions marked with @TODO - [*] Provided justification for your final learner selection - [*] Knitted to HTML successfully - [*] Saved as PDF with correct naming convention